AP HOGESCHOOL ANTWERPEN

Technical Documentation Software

Smart Cane 2025-2026

Weyers Axel · Bosvelt Cederic
Kennes Joyce · Gilles Dario
Taamouti Adnane · Doudouh Yassine

Institution: AP Hogeschool Antwerpen
Supervisors: De Vos Jeroen, Peeters Tom

Table of Contents

TERMS AND ABBREVIATIONS **3**

1. PROJECT SUMMARY **4**

2. INFRASTRUCTURE IMPACT **5**

2.1 HARDWARE 5
2.2 SOFTWARE 5
2.2.1 Licences *5*
2.2.2 Scalability *6*

3. RELEASE PLAN **6**

3.1 HARDWARE 6
3.2 SOFTWARE 6
3.2.1 Build and install *6*
3.2.2 First-run configuration *6*
3.2.3 CI/CD pipeline *7*

4. TECHNICAL DESIGN **7**

4.1 TECHNOLOGY OVERVIEW 7
4.2 APP ARCHITECTURE 8
4.2.1 Class diagram *8*
4.3 PACKAGE STRUCTURE 9
4.4 SCREEN OVERVIEW 10
4.5 NAVIGATION GRAPH 14
4.6 BLE ARCHITECTURE 15
4.7 EMERGENCY / SOS MODULE 17
4.8 NAVIGATION MODULE 17
4.9 BLE TEXT-TO-SPEECH FLOW 20

5. EXTERNAL SYSTEM INTERFACES **21**

5.1 SOFTWARE 21
5.2 DFD COMPONENTS 22
5.3 SOS FLOW 23

6. DATA MIGRATION **23**

7. SECURITY AND AUTHORISATION ROLES **24**

7.1 ANDROID PERMISSIONS 24
7.2 BLE SECURITY 24
7.3 DATA PRIVACY 25
7.4 AUTHORISATION ROLES 25

8. DOCUMENTATION **25**

REFERENCES **26**

Terms and abbreviations

The following table defines all terms and abbreviations used in this document:

Term / Abbreviation	Description
BLE	Bluetooth Low Energy
GATT	Generic Attribute Profile: BLE service / characteristic structure
TTS	Text-to-Speech
osmdroid	Open-source Android map library (OpenStreetMap)
DFD	Data Flow Diagram
APK	Android Package: compiled Android application file
CI/CD	Continuous Integration / Continuous Deployment
GPS	Global Positioning System
UUID	Universally Unique Identifier
JSON	JavaScript Object Notation: lightweight data interchange format
MVVM	Model-View-ViewModel: Android architectural pattern
SmsManager	Android API for sending SMS messages
MapBox	Cloud service for geocoding and route calculation APIs
CCCD	Client Characteristic Configuration Descriptor (BLE notification enable)
SharedPreferences	Android key-value storage for persisting small data
OSRM	Open Source Routing Machine
ADR	Automatic Speech Recognition: Android built-in speech-to-text API
StateFlow	Kotlin coroutine-based observable state holder used in ViewModels
NavHost	Jetpack Navigation component that manages the navigation back stack
HTTPS	HyperText Transfer Protocol Secure: encrypted HTTP used for Mapbox API calls
POST_NOTIFICATIONS	Android runtime permission required to display foreground service notifications on Android 13+

1. Project summary

Safe and independent mobility remains a major challenge for visually impaired people, particularly in environments with uneven infrastructure, dense traffic, and limited accessibility. A traditional white cane offers only ground-level detection, leaving users unaware of obstacles at hip or chest height and providing no digital support for navigation, emergency situations, or environmental awareness.

The Smart Cane project was developed to address this gap. The system combines a Raspberry Pi-based hardware platform with an Android companion application, working together over Bluetooth Low Energy to provide a more complete and independent mobility experience. The hardware handles all on-device sensing, obstacle detection, audio feedback, and AI object recognition. The Android app extends these capabilities by providing turn-by-turn walking navigation, real-time GPS location, emergency SOS dispatch, and remote volume control for the cane's speaker.

This document covers exclusively the software side of the system: the Android companion application and its BLE communication layer with the Smart Cane. The hardware design is documented separately in the Technical Documentation Hardware document, and the CI/CD pipeline in the CI/CD documentation.

2. Infrastructure impact

2.1 Hardware

The hardware infrastructure impact is described in full in the Technical Documentation Hardware document.

From a software perspective, the following hardware is required at minimum:

- An Android smartphone (API level 24 / Android 7.0 or higher) with Bluetooth 5.0 support, GPS, and mobile data.
- The Smart Cane (Raspberry Pi 5) must be assembled and running the Pi firmware before the Android app can establish a BLE connection.

2.2 software

The Android app itself requires no dedicated server infrastructure. All application logic runs on the user's smartphone. The following external cloud services are consumed at runtime:

Service	Purpose	Requirements
Mapbox Geocoding API	Convert typed/spoken destination to GPS coordinates	Active Mapbox access token; internet connection on device
Mapbox Directions API	Calculate walking route with turn-by-turn steps	Active Mapbox access token; internet connection on device
Google Play Services	Real-time GPS via FusedLocationProviderClient	Google Play Services installed on the Android device
Android Telephony (SMS)	Send emergency SMS via SmsManager	Active SIM card with SMS capability in the device

No additional servers need to be purchased or installed for the application.

If the Mapbox Geocoding API is unreachable: raw GPS coordinates are displayed instead of an address. If the BLE connection to the cane is lost: Android on-device TTS replaces the cane speaker for navigation instructions. If there is no internet: osmdroid TilePackager can pre-cache map tiles for the target region (Dar es Salaam) on the device.

2.2.1 Licences

Library / Services	Licence	Cost
Kotlin	Apache 2.0	Free
Jetpack Compose / Jetpack	Apache 2.0	Free
osmdroid	Apache 2.0	Free
Room KTX	Apache 2.0	Free
Google Play Services	Google APIs Terms of Service	Free
Mapbox Geocoding API	Mapbox Terms of Service	Free tier available, commercial use requires paid plan above usage limits
Mapbox Directions API	Mapbox Terms of Service	Free tier available; commercial use requires paid plan above usage limits

Navigation Compose	Apache 2.0	Free

2.2.2 Scalability
The app is designed for single-user, single-cane deployment. Each installation manages one BLE connection to one Smart Cane. No shared backend is required. Distribution of the APK to additional users is handled by the CI/CD pipeline (see section 4) or manual installation.

3. Release plan

3.1 Hardware

Prerequisite for the Android app, the Smart Cane hardware must be fully assembled and the Pi firmware must be running before Android BLE pairing can be tested.

3.2 Software

The following steps describe the full Android app build, installation, and first-run setup.

3.2.1 Build and install

1. Clone the repository from GitLab: git clone ssh://git@gitlab.apstudent.be:8081/bachelor-it/blended-intensive-project/25-26/smart-stick.git
2. Open the project in Android Studio (Hedgehog 2023.1.1 or later).
3. Verify all dependencies resolve: ./gradlew dependencies
4. Build the debug APK: ./gradlew assembleDebug
5. Run unit tests: ./gradlew test
6. Deploy to a connected Android device: ./gradlew installDebug

The Mapbox access token is currently hardcoded in NavigationMapScreen.kt. For production, this token should be moved to local.properties and accessed via BuildConfig.

3.2.2 First-run configuration

On first launch, the system will request the following runtime permissions. All must be granted for full functionality: Bluetooth Scan, Bluetooth Connect, Location, Send SMS, Microphone, Read Contacts, Post Notifications. If Post Notifications is denied, the BLE foreground service notification will not be visible but the service will still run.

1. Grant all requested runtime permissions: Bluetooth Scan, Bluetooth Connect, Location (Fine), Send SMS, Microphone, Read Contacts.
2. Open Settings: enter the emergency contact phone number and the default SOS message text.
3. Open Pairing: scan for BLE devices and connect to SmartStick_BLE (ensure the Smart Cane is powered on).
4. Verify BLE connection is active (connection status shown in the pairing screen).

3.2.3 CI/CD pipeline

The project includes a GitLab CI/CD pipeline for automated build and deployment. This pipeline is documented in a separate CI/CD document. In short, every push to the main branch triggers an automated Gradle build and test run on the Debian server GitLab runner.

4. Technical design

This section describes the Android app software architecture, technologies used, and the key design decisions.

4.1 Technology overview

Technology	Application	Notes
Kotlin 2.0.21	Android App language	primary Android language
Jetpack Compose (BOM 2024.09)	UI framework	Declarative; all screens built with Composables
Material 3	UI design system	Accessible components; colour, typography, shapes
Navigation Compose 2.8.8	Screen routing	Type-safe routes; NavHost with sub-graphs
ViewModel / StateFlow	State management	MVVM pattern; reactive UI driven by StateFlow
osmdroid 6.1.20	Map rendering	OpenStreetMap tiles; no Google Maps SDK dependency
Google Play Services Location 21.3.0	GPS	FusedLocationProviderClient for real-time coordinates
Mapbox Geocoding API	Address resolution	REST; converts text/coords to readable address
Mapbox Directions API	Route calculation	REST; walking profile; turn-by-turn steps + geometry
Android SpeechRecognizer	Voice destination input	Built-in ASR; no external dependency
Android TTS	Fallback audio output	On-device TTS when BLE is disconnected
Android SmsManager	SOS SMS dispatch	Direct SMS via Android Telephony; no third-party
SharedPreferences	Emergency contact storage	Key-value; stores emergency_contact and emergency_message
Room KTX 2.8.4	ORM (dependency present)	Included in build; not actively used for data storage yet
Coroutines (via Lifecycle)	Async operations	viewModelScope for BLE callbacks and GPS requests
compileSdk 36 / minSdk 24	Build configuration	Targets Android 7.0+; compiled against latest SDK
MockK 1.13.8	Unit testing	Kotlin-first mock library for ViewModel tests

coil-compose 2.7.0	Image loading	Included as dependency; not actively used in current screens

4.2 App architecture

The app follows the Model-View-ViewModel (MVVM) pattern.

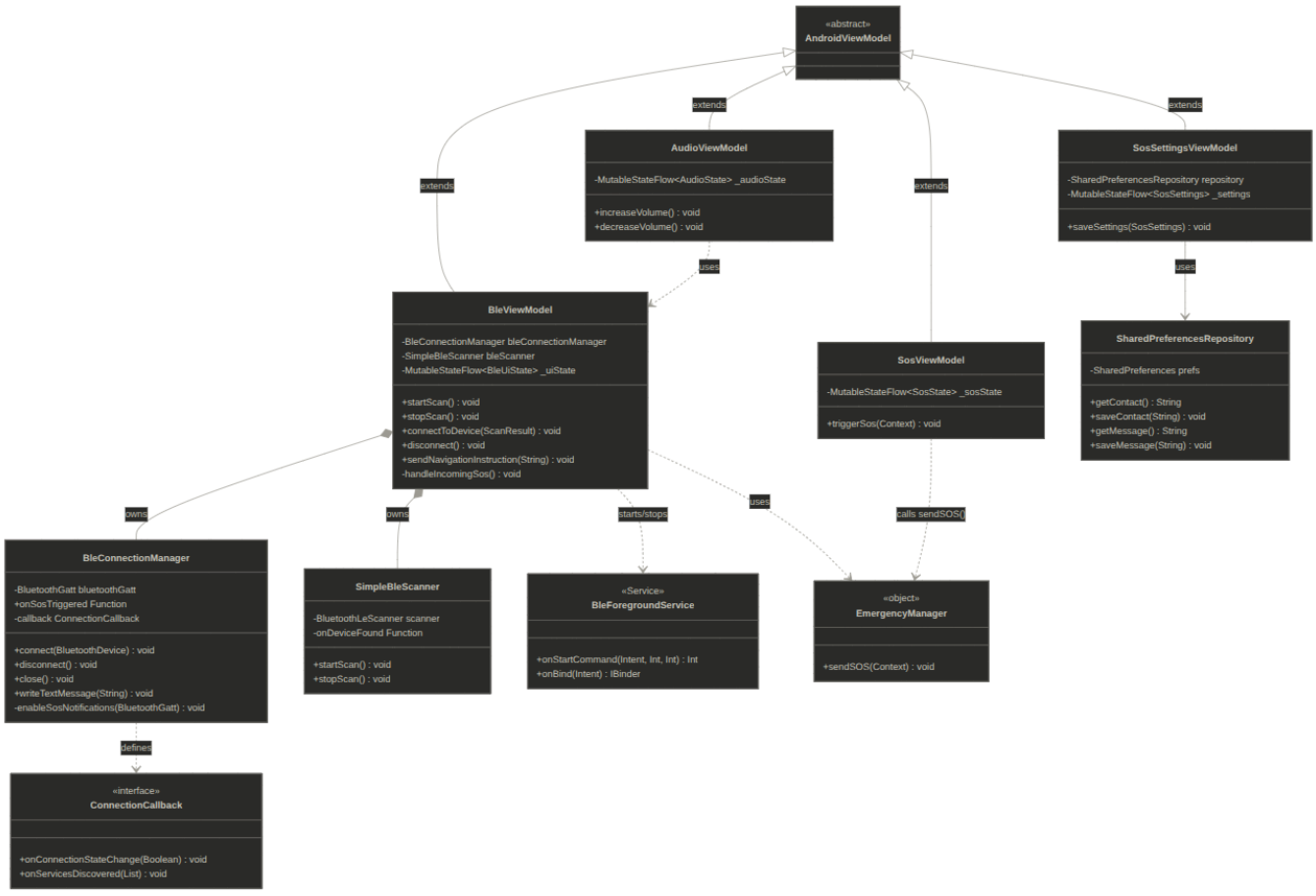
Model: data classes (BleUiState, AudioState, SosSettings, SosState) and repositories (SharedPreferencesRepository, ContactRepository).

View: Composable screens (MainMenuScreen, NavigationMapScreen, SosScreen, etc.) observe StateFlow and render reactively.

ViewModel: BleViewModel, AudioViewModel, SosViewModel, SosSettingsViewModel hold and expose UI state via StateFlow; survive screen rotations.

4.2.1 Class diagram

The class diagram below shows the key classes of the Android application and their relationships. The four ViewModel classes inherit from AndroidViewModel and expose UI state via StateFlow. BleViewModel is the central orchestrator: it owns the BleConnectionManager and SimpleBleScanner instances through composition, and routes incoming SOS signals from the Pi to EmergencyManager. SosSettingsViewModel persists emergency contact data through SharedPreferencesRepository. BleForegroundService is started and stopped by BleViewModel to keep the GATT connection alive when the app is in the background.



Generated with Anthropic Claude Sonnet 4.6 used on may 24 2026.

Legend:
(▷) denote inheritance
(◆) denote composition
(– –>) denote dependency
<<interface>>, <<Service>>, and <<object>> indicate the role of the class within the Android framework.

4.3 Package structure

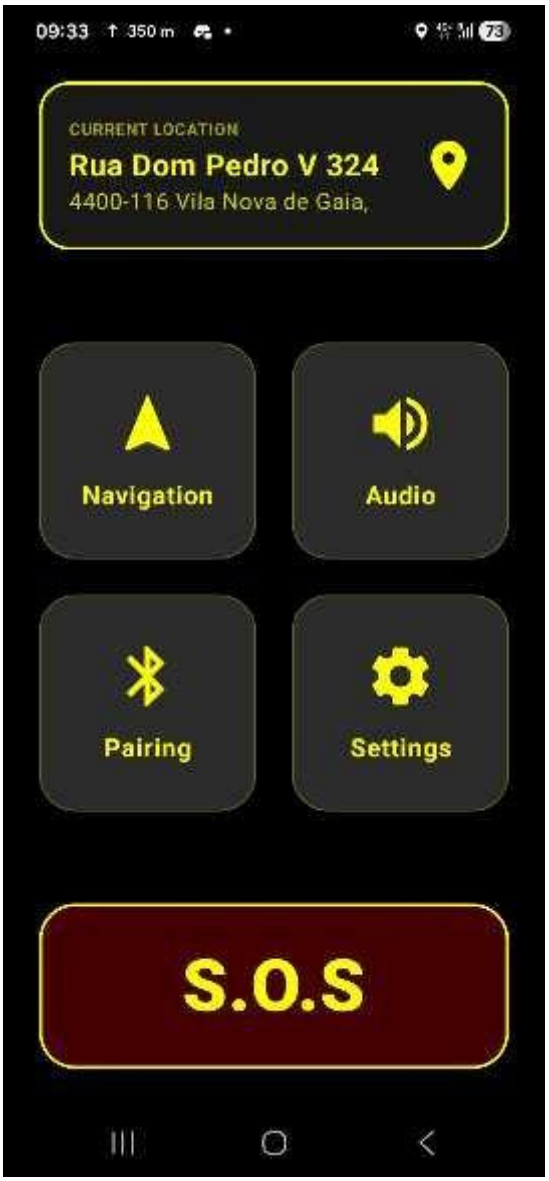
Package	Key classes	Responsibility
logic/	BleConnectionManager, BleViewModel, BleForegroundService, SimpleBleScanner, AudioViewModel, EmergencyManager, ContactRepository, SharedPreferencesRepository, SosSettingsViewModel, SettingsRepository	BLE management, ViewModels, background service, SOS logic, data persistence
model/	BleUiState, AudioState, SosSettings, SosState, SosViewModel, SearchViewModel, factory classes	UI state models and ViewModels
ui/screens/	MainMenuScreen, SearchScreen, ConfirmDestinationScreen, WalkingRouteScreen, NavigationMapScreen, AudioScreen, DeviceListScreen, SosScreen, SosContactSettingsScreen	Composable screen implementations
ui/	AppNavigation, NavigationGraph, SosGraph, DeviceGraph	Navigation host and sub-graph definitions
ui/components/	DeviceComponents	Reusable Composable UI components
ui/theme/	Color, Theme, Type	Material 3 theme configuration
ui/utils/	HapticUtils	Haptic feedback utilities

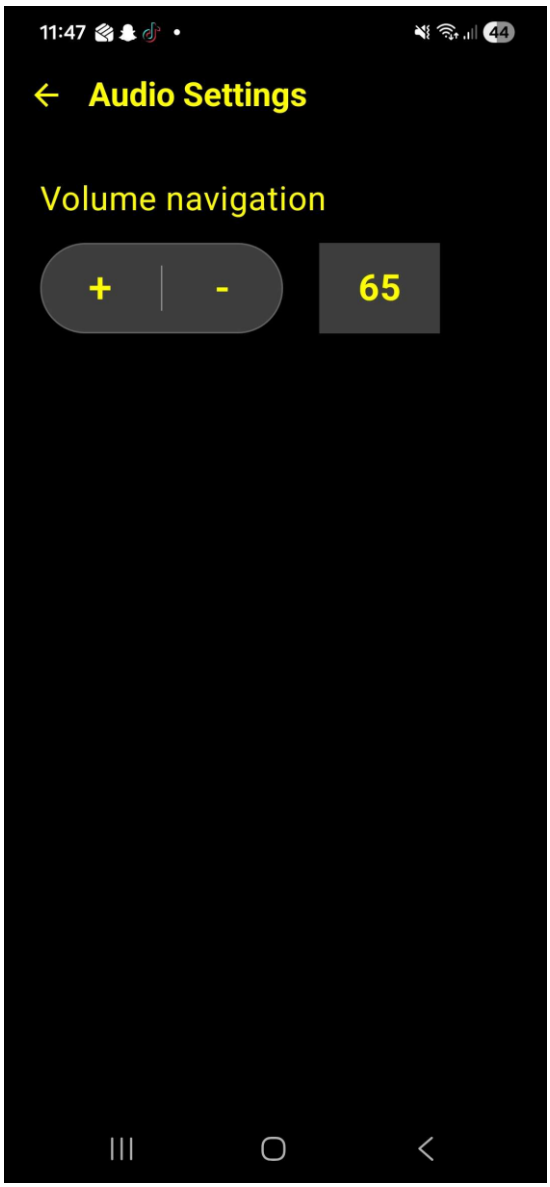
4.4 Screen overview

The table below lists all screens, their navigation route, and function. Screenshots of the key screens follow.

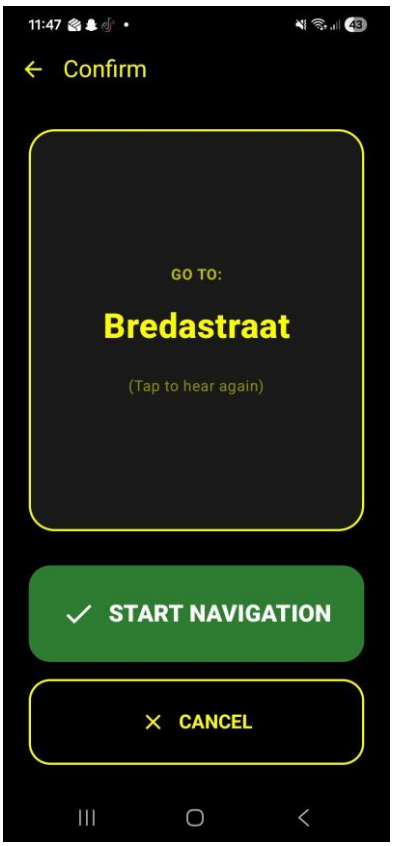
Screen	Route	Function
MainMenuScreen	main_menu	Current GPS location display; buttons: Navigation, Audio, Pairing, Settings, SOS button
SearchScreen	navigation/{lat}/{lon}	Voice or text destination input; Mapbox geocoding
ConfirmDestinationScreen	confirm_destination/{name}/{lat}/{lon}	Show recognised destination; confirm or cancel
WalkingRouteScreen	walking_route/{name}/{lat}/{lon}	Route details: destination, walking time, distance, START NAVIGATION button
NavigationMapScreen	navigation_map/{startLat}/{startLon}/{destLat}/{destLon}/{distance}/{time}	Live osmdroid map; turn-by-turn instruction banner; TIME /DISTANCE footer; sends instructions via BLE
AudioScreen	audio_settings	Single master volume control (+/- buttons, range 0–100); sends JSON volume command over BLE to CHAR_UUID.
DeviceListScreen	device_list	BLE device scan; connect to SmartStick_BLE
SosContactSettingsScreen	contact_settings	Enter and save emergency phone number and default SOS message
SosScreen	sos	Large SOS button, emergency SMS on hold for 3 seconds

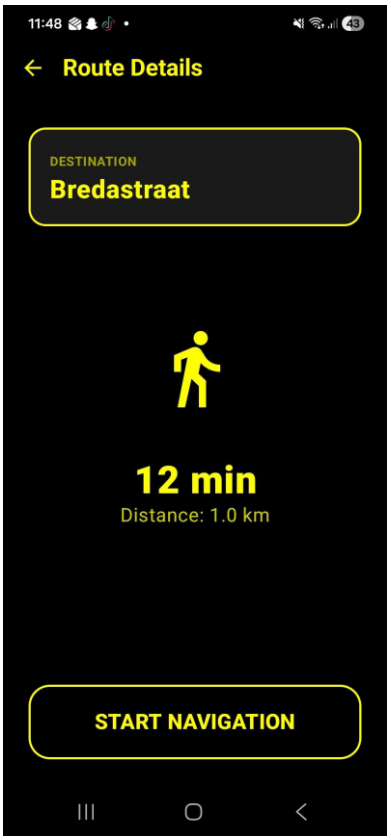
MainMenuScreen and AudioScreen:



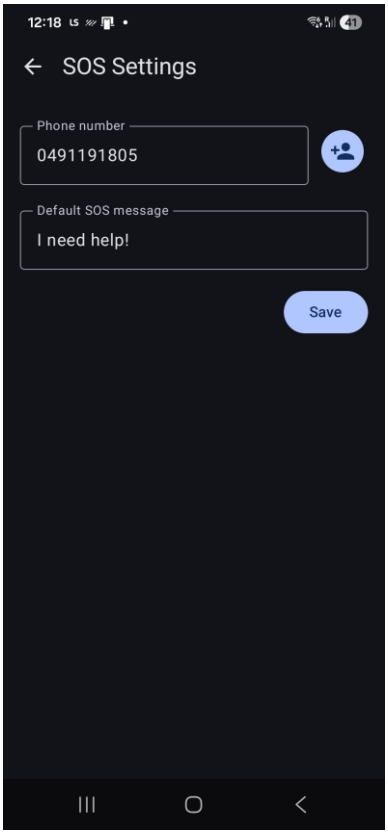


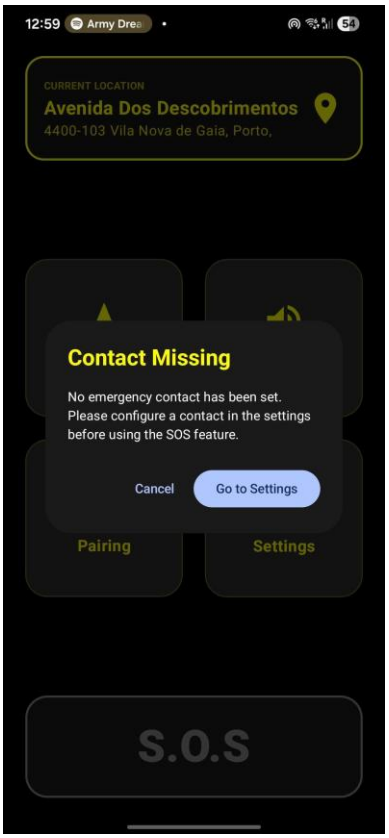
ConfirmDestinationScreen, WalkingRouteScreen (route details) and NavigationMapScreen (live map with turn banner and time/distance footer):





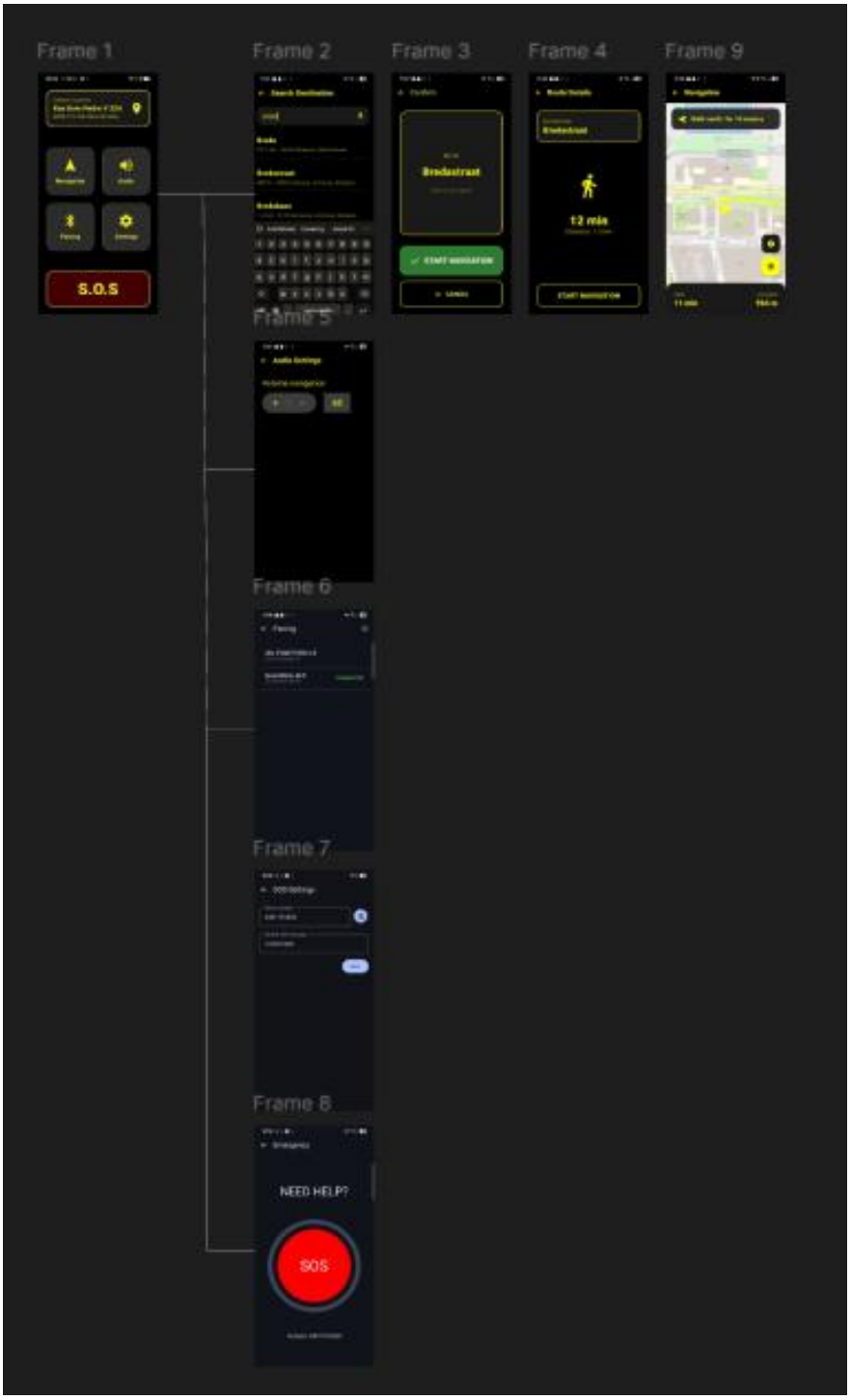
SosContactSettingsScreen, SosScreen, and the 'Contact Missing' dialog shown when SOS is pressed without a configured contact:





4.5 Navigation graph

AppNavigation.kt defines a single NavHost (startDestination = main_menu) with five sub-graphs for modularity:



4.6 BLE architecture

BLE scanning is automatically stopped after 10 seconds to conserve battery.

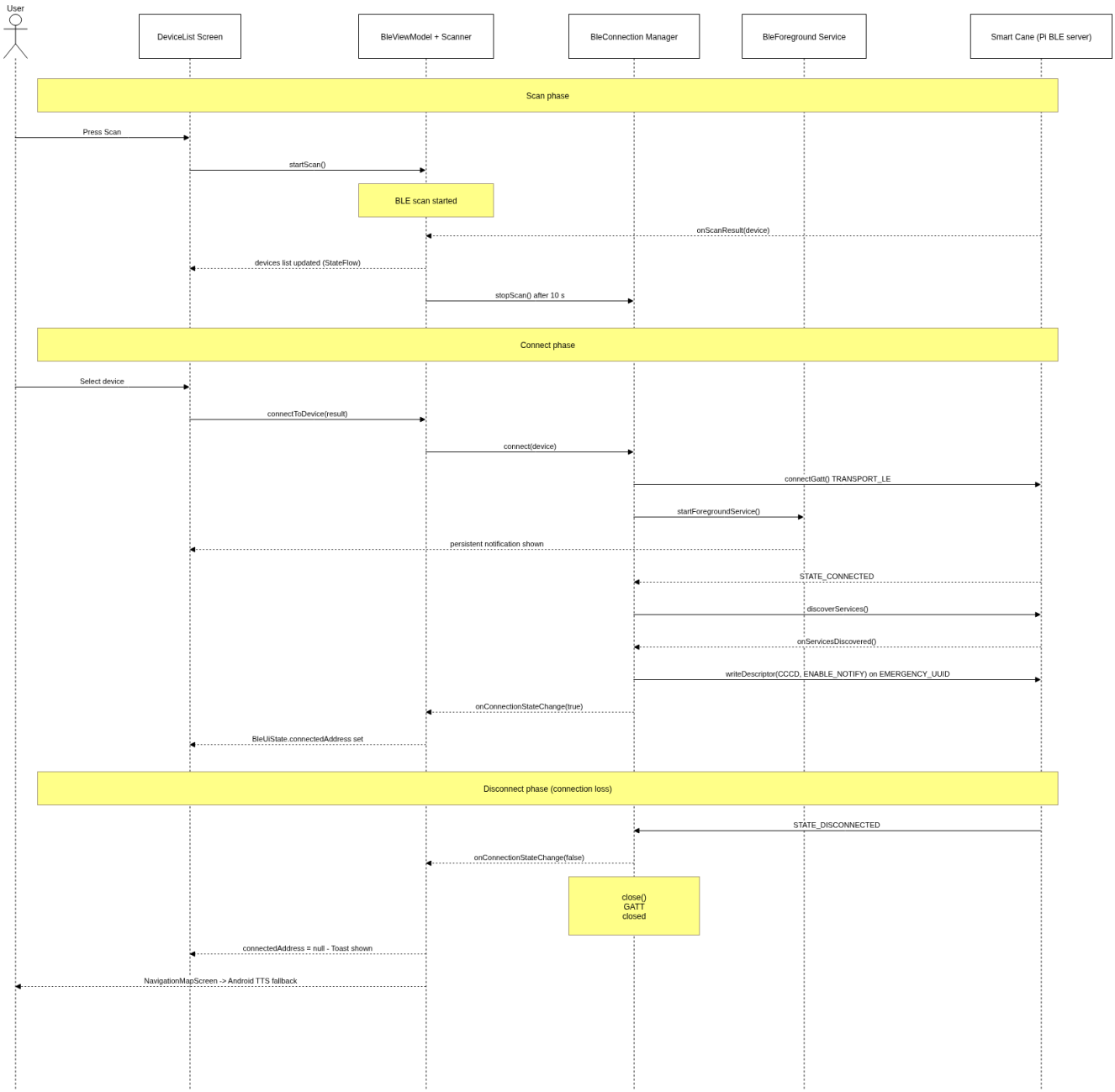
The BLE stack on the Android side is:

Layer	Class / Component	Role
Android BLE API	BluetoothLeScanner, BluetoothGatt, BluetoothGattCallback	Low-level BLE scanning and GATT operations
Connection manager	BleConnectionManager	Manages GATT connection lifecycle; writes to CHAR_UUID, enables SOS notifications on EMERGENCY_UUID
Scanner	SimpleBleScanner	raps BluetoothLeScanner; filters and returns discovered devices
Foreground service	BleForegroundService	Keeps BLE connection alive when app is backgrounded (Android foreground service with FOREGROUND_SERVICE_CONNECTED_DEVICE type)
ViewModel	BleViewModel	Exposes BleUiState (StateFlow); orchestrates scan, connect, disconnect, send; routes incoming SOS trigger to EmergencyManager

GATT characteristic assignment (matches the Pi firmware exactly):

UUID	Name	Direction	Data format
…5679	CHAR_UUID	Android to Pi	TTS: {"type":"tts","value":"Turn left for 120 meters"}. Volume: {"type":"volume","value":"65"}
…567A	EMERGENCY_UUID	Pi to Android	String: "SOS:TRIGGER" notified via CCCD; triggers EmergencyManager.sendSOS()

The following state diagram shows the BLE connection lifecycle on the Android side:

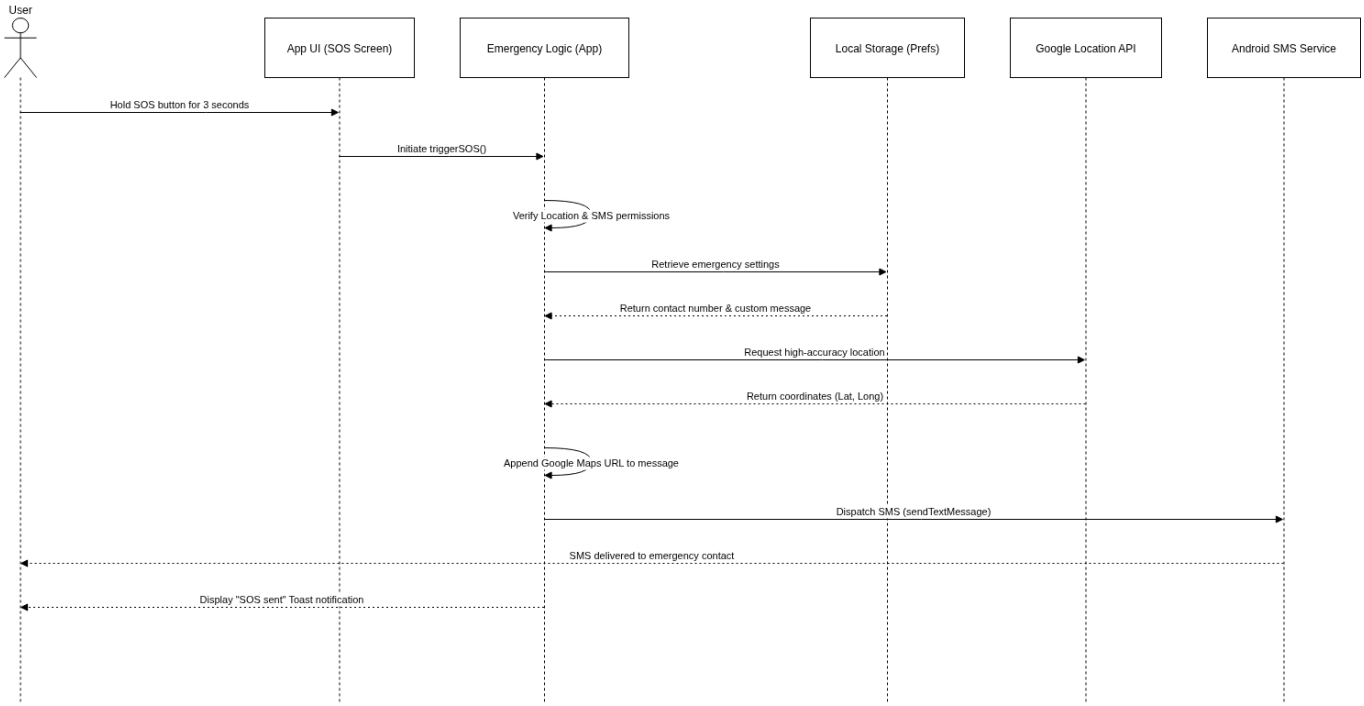


On connection loss, BleConnectionManager calls close() and the BleViewModel updates BleUiState.connectedAddress to null and shows a Toast notification to the user. No automatic reconnection is attempted, the user must return to the Pairing screen and reconnect manually. NavigationMapScreen detects the disconnected state via connectedAddress == null and automatically falls back to Android on-device TTS for navigation instructions.

4.7 Emergency / SOS module

EmergencyManager is implemented as a Kotlin singleton (object) that implements the EmergencyService interface. When sendSOS() is called, it first checks for SEND_SMS and ACCESS_FINE_LOCATION permissions. If either is missing, a Toast is shown and the function returns. It then reads the emergency contact number and custom message from SharedPreferences (emergency_prefs). A high-accuracy GPS fix is requested via FusedLocationProviderClient. Once the location is available, the message is composed as {message}\n\nMy location: https://www.google.com/maps?q={lat},{lon} and dispatched via SmsManager.sendTextMessage() direct Android Telephony, no third-party service. If the GPS request fails or returns null, a Toast informs the user.

Thesequence diagram shows the SOS flow, covering all component interactions between the Android app UI, the emergency logic layer, local storage (SharedPreferences), Google Location API, and the Android SMS Service:



Emergency contact data is stored in SharedPreferences (key: emergency_prefs):

SharedPreferences key	Type	Default
emergency_contact	String	must be configured before SOS can be used)
emergency_message	String	"I need help!"

4.8 Navigation module

The navigation flow begins when the user speaks or types a destination on SearchScreen. The SpeechRecognizer converts the audio to a text string, which is sent to the Mapbox Geocoding API via an HTTPS GET request to resolve coordinates. The recognised destination is shown on ConfirmDestinationScreen, where the user confirms or cancels. On

confirmation, WalkingRouteScreen calls the Mapbox Directions API (walking profile) and parses routes[0].duration and routes[0].distance to display estimated walking time and distance.

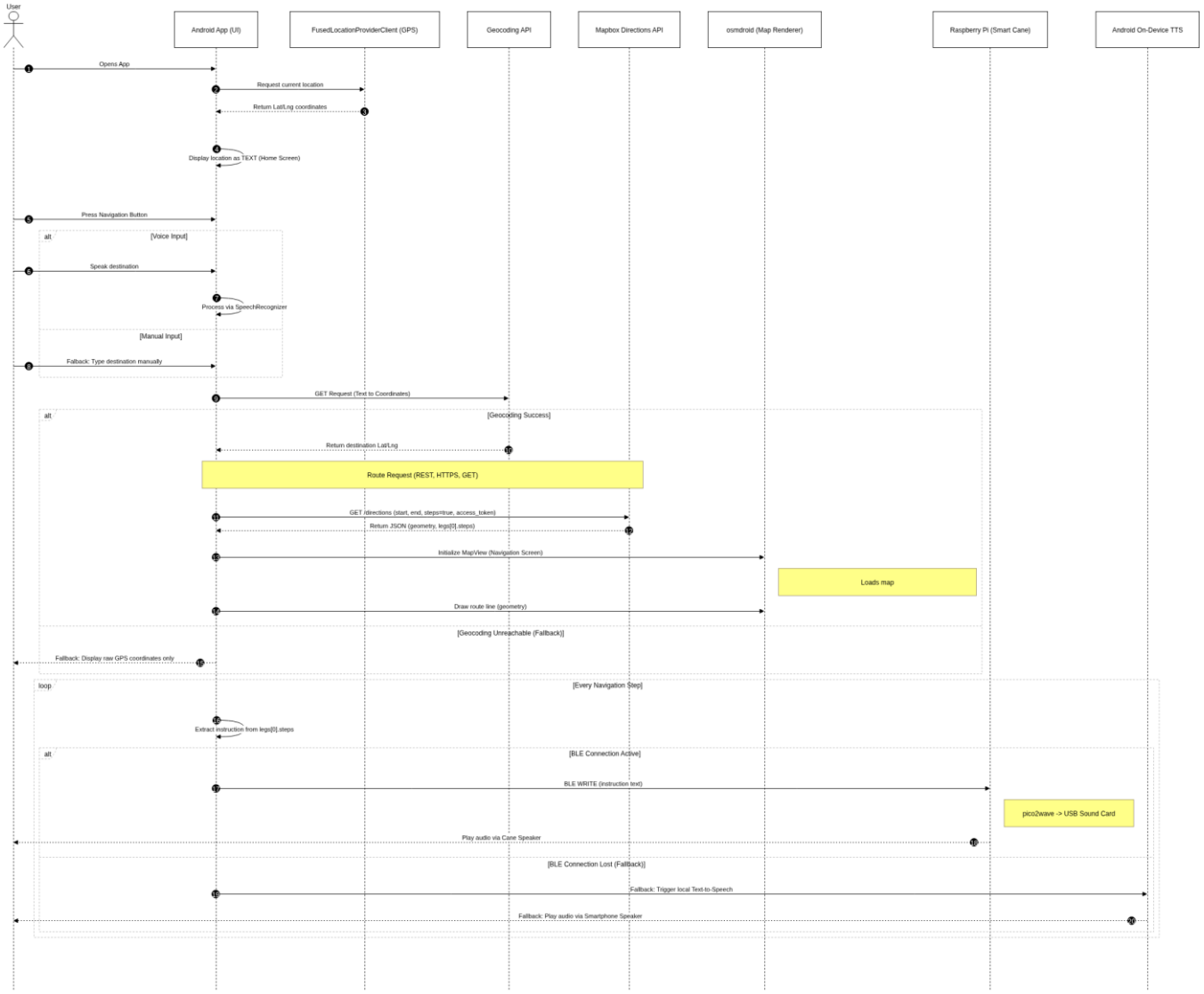
NavigationMapScreen renders the route polyline on an osmdroid map. A LocationListener fires every 5 metres of movement and recalculates the route from the current position. At each navigation step, the instruction string is wrapped in a JSON object {"type":"tts","value":"..."} and sent via BleViewModel.sendNavigationInstruction() as a BLE WRITE to CHAR_UUID. If BLE is disconnected (connectedAddress == null), NavigationMapScreen falls back to Android on-device TTS.

Walking time and distance display logic:

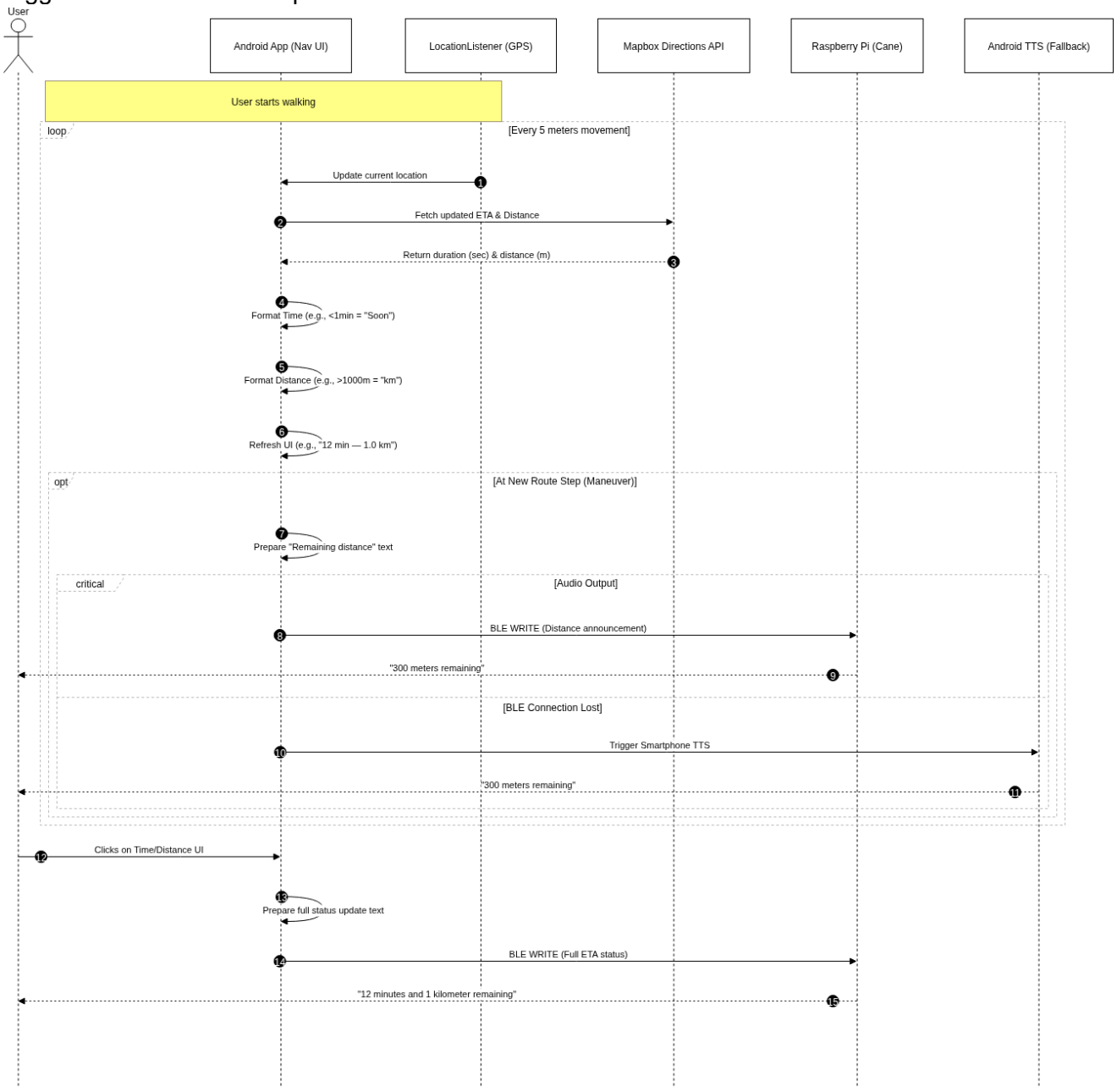
Rule	Behaviour
Update trigger	LocationListener fires every 5 metres of user movement
Time format	In minutes, if < 60 s remaining: display "Soon"
Distance format	< 1000 m: show in metres ("964 m") ≥ 1000 m: show in km with 1 decimal ("1.0 km")
Tap action	Tapping the TIME/DISTANCE footer triggers a TTS announcement of remaining journey

The map also displays a directional arrow that rotates in real time based on the device's orientation sensor (TYPE_ORIENTATION), providing compass-based heading feedback.

The first diagram shows the full navigation flow, covering all component interactions between the Android app, Google Play Services (GPS), the Mapbox Geocoding API, the Mapbox Directions API, the osmdroid map renderer, the Smart Cane (Raspberry Pi) via BLE, and the Android on-device TTS fallback:



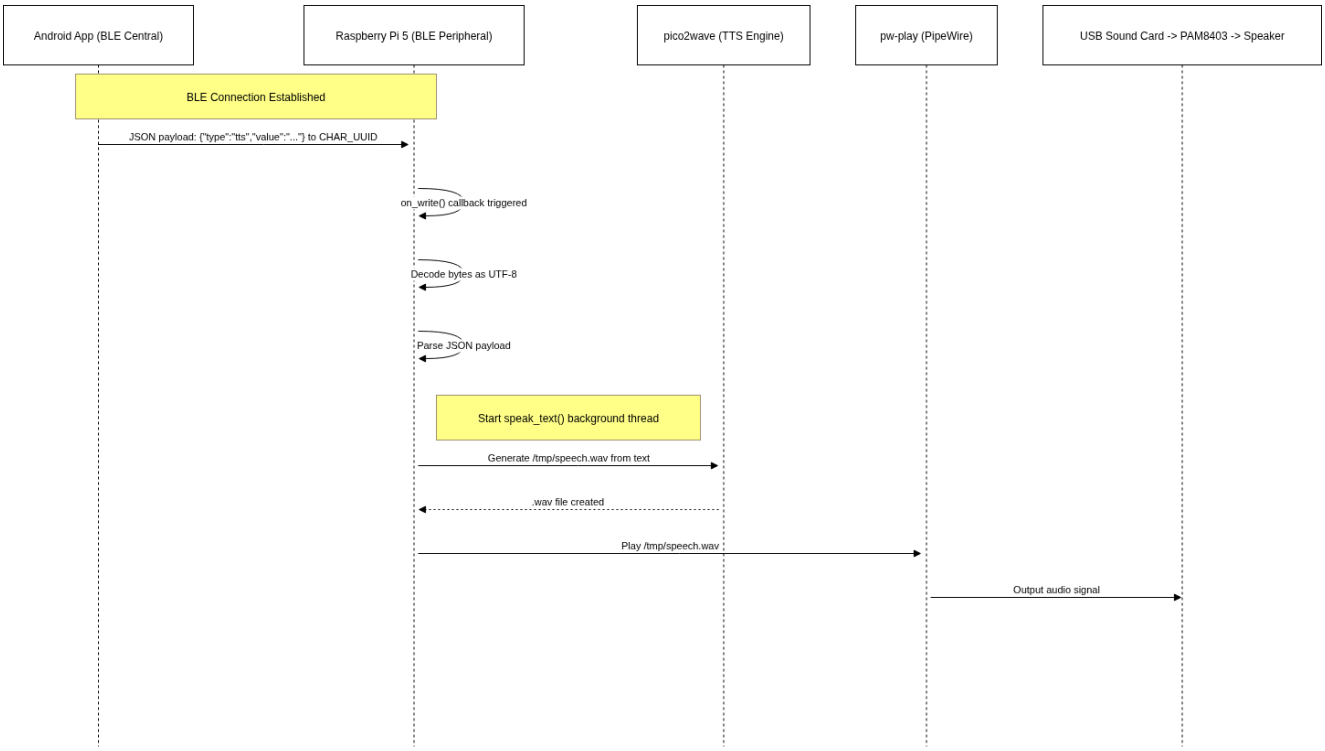
The following sequence diagram shows the walking time and distance update flow during active navigation. It illustrates how the app retrieves a GPS fix every 5 metres, fetches updated ETA and distance from the Mapbox Directions API, formats the values for display, and announces route steps via BLE to the cane speaker. The fallback to Android on-device TTS when the BLE connection is lost is also shown, as well as the manual announcement triggered when the user taps the time/distance UI element.



4.9 BLE Text-to-Speech flow

When a navigation instruction is ready, NavigationMapScreen creates a JSON object {"type":"tts","value":"..."} and calls BleViewModel.sendNavigationInstruction(json.toString()). This calls BleConnectionManager.writeTextMessage(), which writes the UTF-8 encoded bytes to CHAR_UUID using WRITE_TYPE_DEFAULT.

The following sequence diagram shows the complete BLE TTS flow:



More information of the BT on the Raspberry Pi side is in the hardware documentation.

5. External system interfaces

5.1 Software

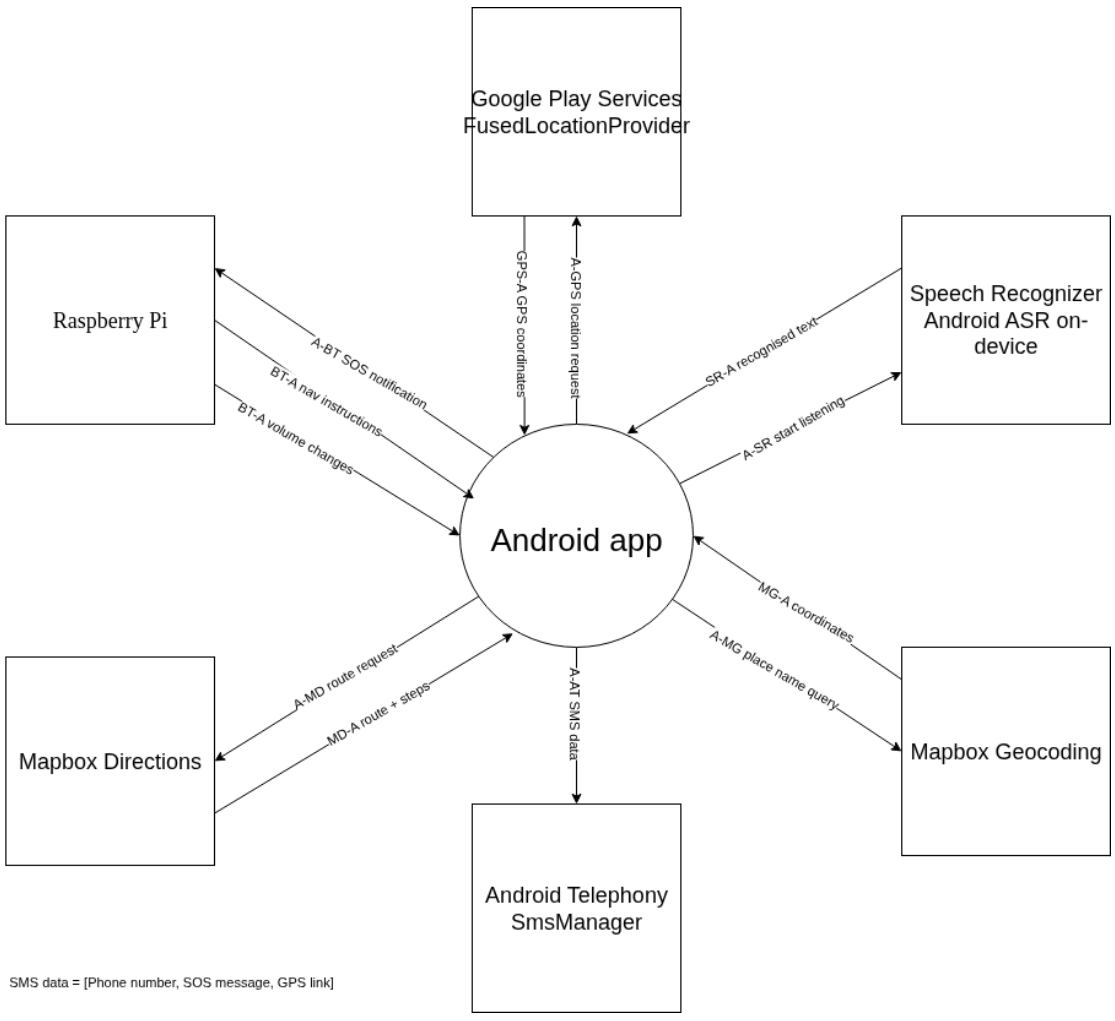
The Android app communicates with the following external systems:

Interface	Protocol / Standard	Direction	Description
SmartCane (Raspberry Pi)	BLE 5.0 / GATT	Bidirectional	Navigation text and volume JSON to Pi (CHAR_UUID WRITE); SOS trigger from Pi (EMERGENCY_UUID NOTIFY)
Mapbox Geocoding API	HTTPS REST / JSON	Android to Mapbox	GET request with text query; response: JSON with coordinates
Mapbox Directions API	HTTPS REST / JSON	Android to Mapbox	GET request with start/end coords and steps=true; response: JSON with geometry and legs[0].steps
Google Play Services (GPS)	Android API	Device-local	FusedLocationProviderClient.getCurrentLocation() and LocationListener

Android SmsManager	Android Telephony API	Android to SMS network	sendTextMessage() with contact number and composed message including Google Maps URL
Android SpeechRecognizer	Android API	Microphone to App	Delivers recognised destination text string to SearchScreen
Android TextToSpeech (TTS)	Android API	App to Speaker	On-device TTS fallback when BLE is disconnected, plays navigation instructions via the phone speaker

5.2 DFD components

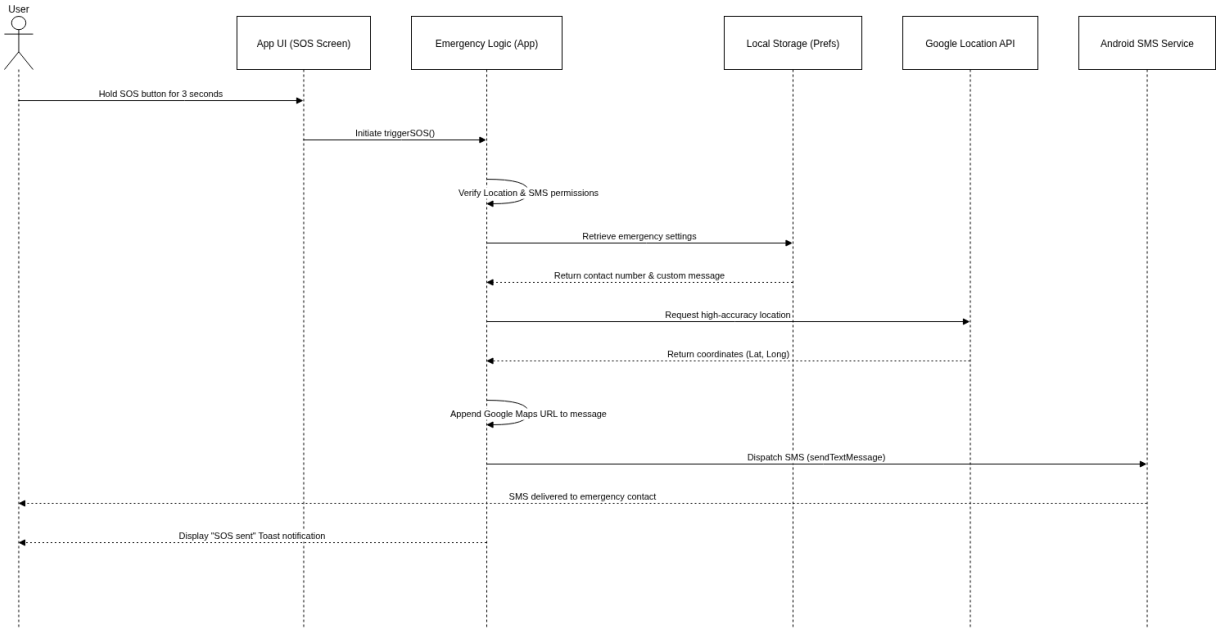
The following Level 0 context diagram shows all data flows between the Android application and its external entities. The app acts as the central process, exchanging data with seven external systems: the Smart Cane hardware over Bluetooth Low Energy, Google Play Services for GPS positioning, the Speech Recognizer for voice input, Mapbox Geocoding for converting place names and coordinates to readable addresses, Mapbox Directions for route calculation, Android Telephony for emergency SMS dispatch



The diagram above captures the following sequence of interactions. The app first requests the current position from Google Play Services, which returns raw GPS coordinates. These coordinates are also sent to Mapbox Geocoding in reverse to retrieve a human-readable address, which is displayed on the home screen as the current location. When the user speaks a destination, the Speech Recognizer returns the recognised text to the app. That text is forwarded to Mapbox Geocoding as a place name query, which returns the destination coordinates. The app then sends both the current coordinates and the destination coordinates to the Mapbox Directions API, which returns the calculated walking route and turn-by-turn instructions. Each navigation step is sent over BLE to the Smart Cane, where the Raspberry Pi converts it to speech and plays it through the speaker. When an SOS is triggered, the app retrieves a fresh GPS fix, composes a message containing the phone number, the SOS text, and a Google Maps link, and passes it to Android Telephony, which dispatches it as an SMS directly to the emergency contact.

5.3 SOS flow

The following sequence diagram shows the full SOS workflow from trigger to SMS delivery:



6. Data migration

The Android app stores minimal persistent user data. All settings are stored in Android SharedPreferences (emergency_prefs):

- emergency_contact (String): the emergency phone number entered by the user.
- emergency_message String): the default SOS message text (default: "I need help!").

No server-side database, cloud storage, or migration scripts are required. There is no data migration between environments.

If the app is uninstalled and reinstalled, SharedPreferences are cleared. The user must re-enter the emergency contact and re-pair the cane via the pairing screen.

Room KTX is included as a Gradle dependency. No Room entities, DAOs, or database migrations exist in the current release. If future features require persistent data (e.g., route history), Room migrations will need to be designed and documented at that point using standard Room migration strategies (auto-migration or migration scripts).

7. Security and authorisation roles

The Android app uses a single-user model with no accounts, passwords, or role-based access control. Security is handled at the OS permission and BLE protocol level.

7.1 Android permissions

All permissions are declared in AndroidManifest.xml. Sensitive permissions are requested at runtime (Android 6.0+):

Permission	When requested	Purpose
BLUETOOTH_SCAN	At pairing screen	Scan for BLE devices
BLUETOOTH_CONNECT	At pairing screen	Connect to SmartStick_BLE
ACCESS_FINE_LOCATION	At navigation / SOS	GPS coordinates for navigation and SOS SMS
SEND_SMS	At SOS trigger	Send emergency SMS via SmsManager
READ_CONTACTS	At SOS settings	Pick a contact from the address book
FOREGROUND_SERVICE_CONNECTED_DEVICE	On BLE connect	Keep BleForegroundService alive in background
WAKE_LOCK	On BLE connect	Prevent CPU sleep during active BLE session

7.2 BLE security

BLE pairing between the Android app and the Smart Cane uses the standard BLE bonding procedure. After bonding, only the paired Android device can connect to SmartStick_BLE. All GATT traffic is transmitted over an encrypted BLE 5.0 link. The Pi side of the BLE configuration is described in the Technical Documentation Hardware document.

7.3 Data privacy

Emergency contact phone numbers and message text are stored locally on the device only (SharedPreferences). They are never transmitted to any cloud service. GPS coordinates are sent to Mapbox APIs for geocoding and routing; Mapbox's privacy policy applies. Camera images (YOLOv8n on the Pi) are processed entirely on the Pi and never transmitted to the Android app or any cloud service.

7.4 Authorisation roles

Role	Access	Authentication
Physical device owner (user)	All app features: navigation, SOS, BLE pairing, volume, settings	None, physical possession of the device implies authorisation
BLE peripheral (Smart Cane)	Writes to CHAR_UUID (Pi receives TTS/volume), notifies EMERGENCY_UUID (SOS trigger)	BLE bonding credential stored on both devices after initial pairing
Developer	APK build and deployment via CI/CD pipeline; SSH access to Raspberry Pi over local network for firmware updates	GitLab credentials (CI/CD); Raspberry Pi OS credentials (SSH)

8. Documentation

All source code is hosted on GitLab at gitlab.apstudent.be. Comments in the Kotlin source code are in Dutch and in English. A future cleanup pass to standardise to English

The following documents are provided as deliverables:

- Technical Documentation Hardware: hardware design, GPIO wiring, sensor modules, Pi firmware, BLE GATT server configuration.
- Technical Documentation Software (this document): Android app architecture, BLE central implementation, navigation and SOS modules, security.
- CI/CD Documentation : GitLab pipeline configuration, Debian runner setup, build and deploy procedures.
- User Guide: end-user instructions for operating the Smart Cane and Android companion app.

References

Android Developers. (2024). Bluetooth low energy overview. Google. https://developer.android.com/develop/connectivity/bluetooth/ble/ble-overview

Android Developers. (2024). Jetpack navigation for Compose. Google. https://developer.android.com/develop/ui/compose/navigation

Android Developers. (2024). SmsManager. Google. https://developer.android.com/reference/android/telephony/SmsManager

Google. (2024). FusedLocationProviderClient. Google Developers. https://developers.google.com/android/reference/com/google/android/gms/location/FusedLocationProviderClient

Mapbox. (2024). Directions API. https://docs.mapbox.com/api/navigation/directions/

Mapbox. (2024). Geocoding API. https://docs.mapbox.com/api/search/geocoding/

osmdroid contributors. (2024). osmdroid wiki. GitHub. https://github.com/osmdroid/osmdroid/wiki

Ultralytics. (2023). YOLOv8 documentation. https://docs.ultralytics.com/models/yolov8/